



DeepDesc: integrating retrieval-augmented generation with large language models for smart contract vulnerability detection

Tao Tan¹ · Xiao Chen¹

Received: 15 July 2025 / Accepted: 10 November 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

With the rapid advancement of blockchain technology, the security of smart contracts has become increasingly critical. Most existing automated tools focus on detecting vulnerabilities based on fixed patterns, making it difficult to identify complex business logic flaws—over 80% of which remain undetected. To address this gap, we propose DeepDesc, a novel framework for smart contract vulnerability detection that integrates Retrieval-Augmented Generation (RAG) with large language models (LLMs) and deep learning. DeepDesc employs GPT-3.5-turbo enhanced by RAG to generate high-quality code descriptions using 1,201 real-world vulnerability reports from Code4Rena. These descriptions, combined with the code, are used to train a deep learning classifier using Multimodal Low-rank Bilinear (MLB) pooling. Our method effectively handles both machine-auditable bugs (MABs) and machine-unauditable bugs (MUBs). Experimental results show that DeepDesc achieves an average F1 score of 1.00 for MABs and 0.90 for MUBs, significantly outperforming existing techniques.

Keywords Smart contracts · Vulnerability detection · Large language models · Deep learning · Multimodal fusion

Communicated by: Sheng Wen, Chunyang Chen and James Zheng

This article belongs to the Topical Collection: *Special Issue on When Software Security Meets Large Language Models: Opportunities and Challenges*.

✉ Xiao Chen
Xiao.Chen@newcastle.edu.au

Tao Tan
tao.tan@newcastle.edu.au

¹ School of Information and Physical Sciences, University of Newcastle, Callaghan, Australia

1 Introduction

Since the introduction of Ethereum, smart contracts have become a key application based on blockchain technology (Buterin et al. 2014). Due to their transparency and immutability, smart contracts provide a programmable and automated solution for executing transactions. As the Ethereum blockchain manages a vast amount of digital assets, identifying and rectifying vulnerabilities in smart contracts has become crucial.

Various tools (Feist et al. 2019; Tsankov et al. 2018; Tikhomirov et al. 2018; Luu et al. 2016; Jiang et al. 2018) have been developed to automate the security auditing of smart contracts. These tools typically focus on vulnerabilities with specific control flow or data flow patterns, such as Delegate Call and Timestamp Dependency. However, Zhang et al. (2023) indicate that smart contract vulnerabilities exploited by hackers primarily stem from business logic flaws, with over 80% of vulnerabilities undetectable by existing tools. Jiao et al. (2024) also highlight the challenges in detecting complex business logic vulnerabilities. Consequently, there is an urgent need for innovative approaches to address these threats.

For business logic vulnerabilities that traditional detection tools cannot identify, David et al. (2023) demonstrated that directly using GPT-4 for high-level “Yes/No” vulnerability queries results in a precision of only 4%. The increasing use of LLMs in security-critical applications has raised awareness of their susceptibility to both misuse and external threats (Deng et al. 2025a, b, c, d). To address this issue, researchers have begun exploring methods that combine LLMs with other techniques. Sun et al. (2024) proposed GPTScan, which integrates LLMs with static analysis; Hu et al. (2023) developed an LLM-based framework that detects vulnerabilities through multi-round dialogues; Shou et al. (2024) introduced LLM4Fuzz, combining LLMs with fuzzing. Although these approaches show promise, they still face certain limitations (Zhou et al. 2025; Zhao et al. 2025). For instance, they may be less effective in handling complex business logic vulnerabilities or struggle to generate high-quality vulnerability descriptions. Notably, deep learning-based detection methods are also vulnerable to evasion techniques such as adversarial or obfuscated inputs, highlighting the importance of robustness in security detection frameworks (Chen et al. 2019; Wan et al. 2024).

To bridge the gap between the powerful language understanding capabilities of LLMs and the specific needs of the smart contract domain, we propose an innovative approach called DeepDesc, which combines a RAG-integrated LLM model with deep learning techniques. Ullah et al. (2024) have shown that providing examples in prompts can enhance the performance of LLMs. We leveraged a dataset of 1201 real-world vulnerability reports from Code4Rena to implement a Retrieval-Augmented Generation (RAG) system. This system dynamically retrieves relevant vulnerability reports based on semantic similarity to the input code and incorporates them into the context provided to the LLM. By augmenting the LLM’s input with domain-specific, contextually relevant information, we enhance its ability to generate precise and pertinent descriptions of smart contract vulnerabilities (Lewis et al. 2020). The generated descriptions include both functional explanations of the code and potential security risks. Subsequently, we used these codes and their corresponding descriptions as training data for a deep learning model. We employed Multimodal Low-rank Bilinear pooling (MLB) to fuse the code and description modalities, and trained a vulnerability detection model capable of binary classification of the inspected code based on specific vulnerabilities.

Our research also specifically addresses the vulnerability classification framework proposed by Zhang et al. (2023), which categorizes smart contract vulnerabilities into machine auditable bugs (MABs) and machine unauditable bugs (MUBs). They distinguish these two categories based on their detectability by existing automated tools. MABs typically involve general security issues with fixed patterns, such as control flow hijacking or reentrancy attacks, which can be easily identified by existing analysis tools using predefined rules. In contrast, MUBs cannot be detected by existing tools as they often involve complex business logic or project-specific implementations, requiring domain knowledge from developers to identify. These MUBs account for approximately 80% of vulnerabilities. Table 1 presents the vulnerabilities selected for testing. Among the MABs, we focus on four common types: delegate calls, integer overflow, timestamp dependency, and reentrancy attacks. The datasets for these four types of vulnerabilities are relatively abundant. For MUBs, we selected three complex types as research subjects: Price Oracle Manipulation, atomicity violation, and inconsistent state updates. These three types of vulnerabilities are among the more frequently occurring cases within MUBs. This classification is motivated by the fact that MABs typically have more datasets available, which is advantageous for deep learning methods and thus suitable for verifying the effectiveness of our approach when ample data is available. MUBs, on the other hand, are the primary factors leading to real-world attacks, and detecting these vulnerabilities will test the practical contributions of our method.

Our experiments utilized multiple datasets to evaluate DeepDesc's performance. First, we used secure code from Top200 (MetaTrustLabs 2023b), comprising the top 200 smart contracts by market capitalization, to assess DeepDesc's ability to identify vulnerable code. For MABs, we employed vulnerable smart contracts collected by Qian et al. (2023). These contracts were sourced from three different origins: the Ethereum platform (accounting for over 96%), GitHub repositories, and blog posts analyzing contracts. As for MUBs, we obtained relevant vulnerable smart contracts from web3bugs (Zhang 2023) and DefiHacks (2023). This diverse selection of datasets ensures our evaluation covers a wide range of smart contract vulnerabilities from various sources and types.

Our experimental results show that DeepDesc significantly outperforms current state-of-the-art tools in smart contract vulnerability detection. For MABs, the average F1 score across four types of vulnerabilities reached 1.00, while for MUBs, the average F1 score across three types of vulnerabilities was 0.90. To examine the effectiveness of RAG, we compared various training approaches: training based on code descriptions generated by baseline LLMs, and training based on code descriptions generated by LLMs fine-tuned with vulnerability reports. The results indicate that training on descriptions and code generated by RAG-enhanced LLMs achieved the best performance.

Table 1 Tested vulnerabilities

Root Cause	Sub-Cause
Machine auditable bugs(MABs)	Reentrancy
	Integer Overflow/Underflow
	Delegatecall Vulnerability
	Timestamp Dependency
Machine unauditable bugs(MUBs)	Price Oracle Manipulation
	Atomicity Violations
	Inconsistent State Updates

To the best of our knowledge, it is the first exploration to introduce a RAG-integrated LLMs and deep learning based framework for smart contract vulnerability detection. The main contributions of our paper can be summarized as follows:

1. We propose an innovative smart contract vulnerability detection framework, Deep-Desc, that combines the code understanding capabilities of RAG-integrated LLMs with the pattern recognition advantages of deep learning. This approach not only enhances detection accuracy but also generates detailed code descriptions, potentially offering developers more interpretable insights into vulnerabilities.
2. We conduct comprehensive experimental research on both MABs and MUBs, demonstrating the effectiveness of our approach across a wide range of existing smart contract vulnerability datasets.

We organized the rest of this paper as follows: Section 2 provides essential background information, Section 3 details our approach, and Section 4 presents and analyzes the experimental results. In Section 5, we discuss the limitations of the method and potential directions for future research. Lastly, Section 6 summarizes related work, and Section 7 presents the conclusions.

2 Background and Motivation

In this section, we will explore the types of vulnerabilities in smart contracts that could lead to significant financial losses. We will describe how these vulnerabilities manifest and provide a motivating example.

2.1 Vulnerabilities in Smart Contracts

With the rapid development of blockchain technology, the taxonomy of smart contract vulnerabilities is also evolving rapidly. Chu et al. (2023) categorized vulnerabilities into Solidity language level, Ethereum blockchain level, and EVM level. Today, most of these vulnerabilities are well known to developers and can be effectively detected by existing tools. Recently, Zhang et al. (2023) collected manually discovered smart contract vulnerabilities from the renowned Code4rena (2024) competition, classifying them into two main categories: MABs (17 types) and MUBs (7 types). They found that over 80% of vulnerabilities are difficult for existing methods to detect (machine unauditable). This is because MABs often have fixed data or control flows, and existing static and dynamic analysis tools can effectively help developers identify these vulnerabilities during the project development phase. Deep learning-based methods can automatically learn from historical data. Current vulnerability detection tools utilizing deep learning mostly use Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Opcode as training data. However, these methods have limited effectiveness in dealing with MUBs. Therefore, there is an urgent need for a method that can understand code semantics and effectively detect MUBs.

2.2 Limitations of Direct LLMs Usage

We evaluated LLMs' ability to directly detect specific vulnerabilities. We selected two MABs (Reentrancy [RE] and Timestamp Dependency [TD]) and two MUBs (Price Oracle Manipulation by AMM [PM] and Approval Not Cleared [ANC]), with 10 vulnerable code samples for each type. Using a role-oriented prompting method (as shown in Fig. 1), we questioned GPT-3.5 ten times to minimize the impact of randomness on the responses. The results, as illustrated in Table 2, show accuracy rates of 75% for RE and 78% for TD (MABs), compared to only 15% for PM and 19% for ANC (MUBs). These findings suggest that LLMs face difficulties in directly identifying MUBs.

2.3 Motivation

We will use two real-world examples to illustrate the difference between MABs and MUBs.

Figure 2 demonstrates an instance of a reentrancy attack. In the withdraw function, funds are sent to the caller by executing the external call `msg.sender.call.value()` before updating the caller's balance. If an attacker re-calls withdraw in their own fallback function, they can recursively withdraw funds before the balance is reset to 0, thus withdrawing amounts exceeding their actual balance multiple times. This vulnerability can already be well detected by existing static tools because it has a fixed code pattern: using `call.value()` to send funds before resetting the balance.

Prompt Template

As an experienced smart contract security auditor, please review the following contract for an ... vulnerability. Answer only with 'yes' or 'no'. [Solidity code]

Fig. 1 Prompt template for vulnerabilities

Table 2 Four types of vulnerabilities directly use GPT-3.5 to perform Yes or No query

Vulnerability Type	Yes (%)	No (%)
Price Oracle Manipulation (PM)	15	84
Approval Not Cleared (ANC)	19	80
Reentrancy (RE)	75	24
Timestamp Dependency (TD)	78	21

```
1 contract PoormansHoneyPot {
2
3     mapping (address => uint) public balances;
4
5     function withdraw() public{
6         assert(msg.sender.call.value(balances[msg.sender]))();
7         balances[msg.sender] = 0;
8     }
9 }
```

Fig. 2 A reentrancy vulnerability

MUBs, on the other hand, are often more diverse. Figure 3 shows a price oracle manipulation vulnerability, where the issue lies in line 11 using `curvePool.get_dy_underlying()` to calculate asset value. This method relies on instant exchange rates, and the code does not employ a Time-Weighted Average Price (TWAP) mechanism to smooth price fluctuations. Attackers could potentially exploit flash loans to temporarily manipulate prices, thereby affecting the function's output and gaining illegal benefits. Understanding and discovering this type of vulnerability requires in-depth knowledge of flash loan mechanisms and how price oracles work. Moreover, the vulnerability involves complex business logic, such as asset valuation and exchange rate calculations, without clear syntactic or structural characteristics. This makes it difficult for static analysis tools to identify, and deep learning tools trained on ASTs and CFGs face similar challenges.

Despite the limitations described in Section 2.2, LLMs demonstrate powerful semantic understanding capabilities in accurately describing code. If we ask LLMs to provide a detailed description of the execution process for this code snippet, their response would likely include statements such as “*calculate asset value in the curve pool,*” “*return the exchange rate for specified tokens,*” and “*liquidity pool used for asset conversion.*” We posit that these descriptive elements may contain key information for identifying such vulnerabilities, potentially enabling deep learning training on LLM-generated code descriptions.

3 Methodology

This section will elaborate on the architectural design of DeepDesc. Figure 4 illustrates the overall workflow of this framework. DeepDesc utilizes Code4Rena vulnerability contest reports and Solidity code samples to generate code descriptions through LLM. These descriptions are used to construct a dataset for training deep learning models. The framework employs K-fold cross-validation to assess the effectiveness of the models in detecting smart contract vulnerabilities. We will detail these steps in the following sections.

```

1  function investedAssets() external view override(BaseStrategy)
    returns (uint256) {
2      uint256 underlyingBalance = _getUnderlyingBalance();
3      uint256 aUstBalance = _getAUstBalance() + pendingRedeems;
4
5      uint256 ustAssets = ((exchangeRateFeeder.exchangeRateOf(
6          address(aUstToken),
7          true
8      ) * aUstBalance) / 1e18) + pendingDeposits;
9      return
10         underlyingBalance +
11         curvePool.get_dy_underlying(ustI, underlyingI, ustAssets
12     );

```

Fig. 3 A price oracle manipulation vulnerability

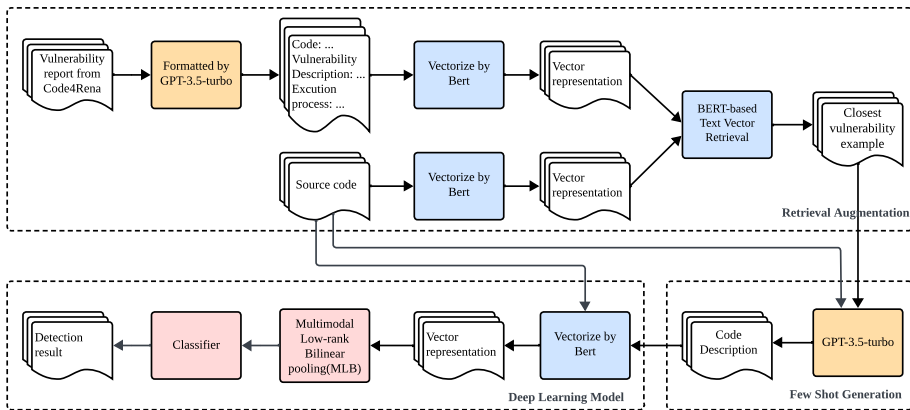


Fig. 4 A high-level overview of DeepDesc

3.1 Preparation of RAG Data Sources

3.1.1 Format Vulnerability Report

RAG is an advanced natural language processing technique that integrates information retrieval and text generation (Lewis et al. 2020). This method enhances the capabilities of large language models by incorporating external knowledge during the generation process. In this study, DeepDesc innovatively utilizes vulnerability reports from the renowned Code4rena (2024) bug bounty competitions as a source of external knowledge. Code4Rena is a decentralized smart contract audit platform that improves blockchain project security through competitive vulnerability reward programs. Security researchers from around the world review code on this platform and submit detailed vulnerability reports. We have collected vulnerability reports covering 255 projects from April 2021 to May 2024, encompassing 1,201 high-risk vulnerability analyses. These analyses, written by auditors from various periods, exhibit diverse stylistic characteristics and predominantly focus on MUBs, which account for approximately 80% of real-world vulnerabilities. To enhance the consistency and usability of these reports, we employed GPT-3.5 (with temperature=1 and top-p=1) to format them into a uniform structure, facilitating subsequent vector matching. Figure 5 illustrates the prompt template.

3.1.2 Vectorize Formatted Vulnerability Report

The vectorization of formatted vulnerability reports transforms textual information into high-dimensional numerical representations, facilitating efficient similarity comparisons and information retrieval. The process begins with preprocessing the original text T by removing special characters and punctuation. A BERT tokenizer then converts the cleaned text into a token sequence $S = [t_1, t_2, \dots, t_n]$. This sequence is input into a pre-trained BERT model, generating a series of hidden states $H = [h_1, h_2, \dots, h_n]$, where each h_i is a high-dimensional vector. The first hidden state h_1 (corresponding to the [CLS] token) is selected as the vector representation v for the entire text. This entire process can be represented as a composite function:

Prompt Template

The following vulnerability report describes a vulnerability in a Solidity contract. Based on the vulnerability report below, help me identify the type of vulnerability, the detailed cause of the vulnerability, the specific execution process, and the vulnerable code.

Please output in the following format without any additional content:

[Type of vulnerability]: ...

[Cause of vulnerability]: ...

[Execution process]: ...

[Vulnerable code]: ...

Below is the vulnerability report you need to process:

[vulnerability report]

Fig. 5 Prompt template for vulnerability reports

$$v = f_{\text{vectorize}}(T) = h1(fBERT(ftokenize(fclean(T))))$$

In this equation, $f_{\text{vectorize}}$ maps the vulnerability report text space to a fixed-dimensional (768-dimensional) real vector space. This method allows for the conversion of text of any length into fixed-length vectors, thereby enabling subsequent analysis and comparison.

3.1.3 Vector Indexing and Similarity Search Using FAISS

We employ the FAISS (Facebook AI Similarity Search) library to construct an efficient vector index, enabling rapid similarity searches. Given a set of vulnerability report vectors $V = \{v_1, v_2, \dots, v_n\}$ generated by a BERT model, where each $v_i \in \mathbb{R}^d$ (in this implementation, $d = 768$), an IndexFlatL2 index based on L2 distance metric is constructed. This index facilitates swift identification of reports most similar to a query vector. For a given query q , it is first transformed into a vector representation v_q using the same BERT model, followed by a k -nearest neighbor search in the index:

$$NN_k(v_q) = \underset{v_i \in V}{\operatorname{argmin}} \sum_{j=1}^k \|v_q - v_i\|_2^2$$

In this equation, $NN_k(v_q)$ denotes the function that returns the k most similar vectors. We set $k = 1$ based on empirical evaluation. Specifically, we conducted preliminary experiments on a validation set of 50 code samples, comparing $k \in \{1, 3, 5\}$. The results showed that $k = 1$ achieved comparable description quality (measured by BLEU score) to larger k values, while avoiding the introduction of noise from less relevant examples in the prompt. Considering our diverse database of 1,201 reports, selecting a single most-similar example provides sufficient domain-specific context for effective few-shot learning.

This method enables efficient similarity searches within large-scale vector collections, thereby rapidly identifying the vulnerability reports most relevant to a given query.

3.2 Few Shot Query

In the task of describing complex real-world smart contracts, LLMs often struggle to accurately capture the essential points (Hu et al. 2023). Research by Ullah et al. (2024) demonstrates that few-shot prompting outperforms zero-shot prompting when utilizing LLMs for vulnerability detection, with the provision of examples significantly enhancing LLM performance. Building on this insight, the present study employs a few-shot prompting approach to optimize the description generation process for smart contracts. Section 3.1.3 has already retrieved the most similar formatted vulnerability reports from the existing database for the code under query. These reports are subsequently used as examples to construct few-shot prompts for the GPT-3.5 model. Through this method, we aim to generate more accurate descriptions of smart contract code. Figure 6 illustrates the specific prompt template employed in this study. This approach not only improves the accuracy of descriptions but also enhances the model's ability to understand and apply domain-specific knowledge, thereby achieving better results in the analysis of complex smart contracts.

3.3 Deep Learning Model

3.3.1 Input Data Processing

The deep learning model of DeepDesc employs a vectorization method similar to that described in Section 3.1.2 to process code and description text. We use the average of the last hidden states of all tokens as the vector representation v for the entire text. This process can be represented by the following composite function:

Prompt Template

You are a smart contract development expert. Please explain the working process of the following code in detail, as thoroughly as possible.

I will provide you with the following information in sequence for your use: [RESPONSE FORMAT], [REFERENCE VULNERABILITY REPORT], and [TASK CODE]. Please note that reference vulnerability report are provided as reference information that you may need. It does not imply that you must answer based on these reference materials. However, if they are helpful, you are welcome to use them.

[RESPONSE FORMAT]

For example, you need to do the following:

The code I am giving you:

```
function sumTokensInPeg( ... ) internal returns (uint256 totalPeg) { ...
}
```

You need to answer like this:

This code defines an internal function named 'sumTokensInPeg', which ...

Here's a detailed explanation of how this function works: ...

[REFERENCE VULNERABILITY REPORT] ...

[TASK CODE] ...

Fig. 6 Prompt template used to generate code descriptions

$$v = f_{\text{vectorize}}(T) = \frac{1}{n} \sum_{i=1}^n h_i(f_{\text{BERT}}(f_{\text{tokenize}}(f_{\text{clean}}(T))))$$

In this equation, $f_{\text{vectorize}}$ maps the vulnerability report text space to a fixed-dimensional (768-dimensional) real vector space. This method comprehensively captures the semantic information of the text by considering the contributions of all tokens.

3.3.2 MLB

The MLB fusion layer is a crucial component of DeepDesc for integrating code embeddings and description embeddings. Given a code embedding $\mathbf{x}_c \in \mathbb{R}^{d_c}$ and a description embedding $\mathbf{x}_d \in \mathbb{R}^{d_d}$, the operation of the MLB layer can be expressed as:

$$\mathbf{z} = \text{MLB}(\mathbf{x}_c, \mathbf{x}_d) = \sigma(\mathbf{U}^T \mathbf{x}_c) \odot \sigma(\mathbf{V}^T \mathbf{x}_d)$$

where $\mathbf{U} \in \mathbb{R}^{d_c \times k}$ and $\mathbf{V} \in \mathbb{R}^{d_d \times k}$ are learnable projection matrices, σ is an element-wise non-linear activation function (ReLU), and $\odot$ denotes the Hadamard product. The output of the MLB layer, $\mathbf{z} \in \mathbb{R}^k$, represents the fused feature representation, encompassing both code structure information and high-level semantic descriptions. This fusion method, through low-rank approximation, effectively captures the interactions between the two modalities while reducing the number of parameters.

3.3.3 Classifier

The classifier layer is a core component of the CombinedEmbeddingClassifier model, transforming the output of the MLB fusion layer into the final classification result. This classifier adopts a multi-layer feedforward neural network structure, including linear transformations, batch normalization, ReLU activation, and Dropout layers. The specific structure is as follows:

1. First layer transformation:

$$\mathbf{h}_1 = \text{ReLU}(\text{BN}_1(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1))$$

where $\mathbf{z} \in \mathbb{R}^{512}$ is the output of the MLB layer, $\mathbf{W}_1 \in \mathbb{R}^{256 \times 512}$, $\mathbf{b}_1 \in \mathbb{R}^{256}$. BN_1 denotes the batch normalization operation. Copy

2. Dropout operation:

$$\mathbf{h}'_1 = \text{Dropout}(\mathbf{h}_1, p = 0.3)$$

3. Second layer transformation:

$$\mathbf{h}_2 = \text{ReLU}(\text{BN}_2(\mathbf{W}_2 \mathbf{h}'_1 + \mathbf{b}_2))$$

where $\mathbf{W}_2 \in \mathbb{R}^{128 \times 256}$, $\mathbf{b}_2 \in \mathbb{R}^{128}$.

4. Applying Dropout again:

$$h'_2 = \text{Dropout}(h_2, p = 0.3)$$

5. Output layer:

$$y = \sigma(w^T h'_2 + b)$$

where $w \in \mathbb{R}^{128}$, $b \in \mathbb{R}$, and σ is the sigmoid activation function.

This multi-layer structure allows the model to learn complex non-linear feature representations. The batch normalization layers help reduce internal covariate shift and accelerate the training process. ReLU activation functions introduce non-linearity, enhancing the model's expressive power. Dropout layers prevent overfitting by randomly dropping neurons during training. Finally, the sigmoid activation function compresses the output to the $[0, 1]$ interval, representing the probability of a specific vulnerability existing in the contract.

3.3.4 Training Process

The training of the DeepDesc model adopts a comprehensive approach to ensure model robustness and generalization capability. We use binary cross-entropy as the loss function:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where y_i is the true label and $\hat{y}_i$ is the model's predicted probability. We employ Adam as the optimizer with an initial learning rate of 0.001. To adapt to the learning dynamics during training, we implement a cosine annealing learning rate schedule:

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t}{T}\pi))$$

Here, η_t is the learning rate at epoch t , and T is the total number of epochs. This strategy allows for rapid learning rate decay in the early stages and slower changes in later stages, facilitating convergence to a more optimal solution. We also incorporate 5-fold cross-validation to evaluate model performance, which not only provides a more reliable performance estimate but also enhances the model's adaptability to different data distributions through multiple training iterations. The validation results from each fold are used for early stopping to prevent overfitting.

3.3.5 Data Augmentation

Given that MUBs (positive samples) are often fewer in number compared to the abundance of secure code (negative samples), this class imbalance could lead to model bias towards the majority class. To address this issue, DeepDesc employs data augmentation for positive samples in the training set. Concretely, oversampling increases the frequency of positive

samples appearing in mini-batches, which mitigates the trivial solution of predicting negatives for most cases—thereby improving recall on rare but security-critical positives.

Let the original dataset be $D = (x_i^c, x_i^d, y_i)_{i=1}^N$, where x_i^c and x_i^d represent code and description embeddings respectively, and $y_i \in \{0, 1\}$ denotes the label. We define $N_+ = |\{i : y_i = 1\}|$ and $N_- = |\{i : y_i = 0\}|$ as the number of positive and negative samples. We oversample the positive samples by generating $\lfloor N_- / N_+ \rfloor - 1$ copies:

$$D_{\text{aug}} = D \cup (x_i^c, x_i^d, y_i)_{y_i=1} \times (\lfloor N_- / N_+ \rfloor - 1)$$

After augmentation, to prevent the model from memorizing the order of repeated samples, we randomly shuffle D_{aug} :

$$D_{\text{final}} = \text{Shuffle}(D_{\text{aug}})$$

We choose straightforward oversampling rather than synthesizing new code/descriptions to avoid semantic drift for MUBs, whose labels rely on nuanced business logic and cross-process semantics. This keeps the minority-class manifold faithful to the original distribution while still rebalancing training signals.

Moreover, due to the limited and heterogeneous nature of MUBs, we adopt 5-fold cross-validation (consistent with the training process) to obtain more stable estimates under small-sample regimes. Cross-validation reduces variance from any single train/validation split, supports early stopping and threshold selection on held-out folds, and provides a fairer assessment of generalization across different MUB categories.

4 Evaluation

In this section, we will use some popular smart contract vulnerability datasets to evaluate our approach. We will first describe the experimental dataset, then present the experimental results, followed by a comparative analysis and a few case studies.

4.1 Dataset

We utilized multiple datasets to ensure a comprehensive evaluation.

4.1.1 MABs

The positive samples for these vulnerabilities were sourced from the research of Qian et al. (2023). They collected vulnerable code from three different sources: the Ethereum platform (accounting for over 96%), GitHub repositories, and blog posts analyzing smart contracts. From this collection, we selected four types of vulnerabilities:

- **Delegate Call:** When a contract uses `delegatecall` to invoke an external contract, it may allow the called contract to modify the state of the caller.
- **Integer Overflow:** When performing operations on integers, exceeding the range (e.g., overflow in addition) can lead to incorrect values.

- **Timestamp Dependency:** If a contract's logic relies on the block timestamp, it may behave unexpectedly due to potential manipulation of the timestamp.
- **Reentrancy Attack:** When a contract's state is not updated during an external call, an attacker can recursively call the same function, leading to multiple withdrawals of funds.

As shown in Table 3, there are 196 contracts with Delegate Call vulnerabilities, 275 contracts with Integer Overflow vulnerabilities, 349 contracts with Timestamp Dependency vulnerabilities, and 273 contracts with Reentrancy vulnerabilities. These contracts all have fixed code patterns. For example, Reentrancy vulnerability code always uses `call.value` to send tokens. Therefore, it can be assumed that traditional symbolic execution tools can detect these vulnerabilities with approximately 100% accuracy. This dataset is used to train and evaluate the effectiveness of DeepDesc in scenarios with larger datasets.

4.1.2 MUBs

The positive samples for these types of vulnerability come from the Web3Bugs (Zhang 2023) and DefiHacks (2023) datasets. Sun et al. (2024) classified the vulnerabilities from both datasets (MetaTrustLabs 2023a, c). Based on this classification, we selected three types that are more challenging for manual auditing and have been frequently exploited in the real world (Zhang et al. 2023), to test the effectiveness of DeepDesc when dealing with complex vulnerabilities with limited datasets:

- **Price Oracle Manipulation:** Attackers gain illegal benefits by manipulating the query results of price oracles. This is usually caused by improper use of price oracle APIs in application contracts.
- **Atomicity Violations:** These occur when multiple business processes concurrently access the same state variables, and developers fail to ensure business-level atomicity. This can lead to attacks such as front-running.
- **Inconsistent State Updates:** In smart contracts with multiple interrelated state variables, updating one variable while forgetting to update or incorrectly updating other related variables. This can result in various consequences ranging from statistical errors to financial losses.

As shown in Table 4, the filtered data includes 138 vulnerability functions for Price Oracle Manipulation, 34 for Atomicity Violations, and 38 for Inconsistent State Updates. This dataset is used to train and evaluate DeepDesc's ability to identify MUBs .

Table 3 MABs dataset

Vulnerability Type	Number of Vulnerable Contracts
Delegate Call	196
Integer Overflow	275
Timestamp Dependency	349
Reentrancy	273

Table 4 MUBs dataset

Vulnerability Type	Number of Vulnerable Functions
Price Oracle Manipulation	138
Atomicity Violations	34
Inconsistent State Updates	38

4.1.3 Non-vulnerable Code

We utilized the Top200 dataset (MetaTrustLabs 2023b), which comprises the top 200 projects by market capitalization, totaling 1,796 contracts considered non-vulnerable. These data were manually curated by MetaTrustLabs and can be used directly without the need for data cleaning. To mitigate the impact of code length on training, as shown in Table 5, we categorized the MUBs dataset by code length into three classes: short files (1-10 lines), medium files (11-30 lines), and long files (more than 30 lines). Based on the distribution of 210 vulnerable samples (41 short files, 120 medium files, and 49 long files), we extracted 500 non-vulnerable functions from the non-vulnerable contracts in the same proportions: 98 from short files, 286 from medium files, and 116 from long files. This dataset is used to train and evaluate DeepDesc's ability to identify negative samples (non-vulnerable code).

4.2 Threshold

The deep learning model of DeepDesc outputs results between 0 and 1, necessitating the establishment of a threshold to distinguish between negative and positive cases. Using the Price Oracle Manipulation vulnerability as an example, we tested the effects of different thresholds, with results shown in Table 6. As the threshold increases, false positives (FP) decrease from 10 to 3 cases, while false negatives (FN) increase from 5 to 10 cases. At a threshold of 0.5, the model achieves a favorable balance: 132 true positives (TP), 5 FP, 495 true negatives (TN), and only 6 FN. Consequently, we chose 0.5 as the threshold for the DeepDesc model, ensuring a high detection rate of 132/138 (95.7%) while maintaining a false positive rate of 5/500 (1%) within an acceptable range.

Table 5 Code length and distribution in datasets

Code Length	Vulnerable Code	Non-vulnerable Code
Short (1-10 lines)	41	98
Medium (11-30 lines)	120	286
Long (30+ lines)	49	116
Total	210	500

Table 6 Results of different thresholds on price oracle manipulation vulnerability

Datasets	Metrics	0.3	0.4	0.5	0.6	0.7	0.8
Negative Sample	TN	490	491	495	495	495	497
	FP	10	9	5	5	5	3
Positive Sample	TP	133	133	132	131	130	128
	FN	5	5	6	7	8	10

4.3 Experiment Results

This section will evaluate the performance of DeepDesc in handling both MABs and MUBs datasets. MABs datasets are typically larger in scale and can be effectively detected by existing tools. We primarily use such datasets to verify the feasibility of DeepDesc's deep learning approach when sufficient data is available. The core objective of DeepDesc is to identify MUBs. Currently, these datasets are relatively scarce, but with the rapid development of blockchain technology, it is expected that more related data will become available in the future. This will significantly enhance the accuracy and practical value of the DeepDesc method.

4.3.1 MABs

This section evaluates DeepDesc's performance in identifying MABs in smart contract datasets. We tested four common vulnerability types: Delegate Call (DC), Integer Overflow (IO), Timestamp Dependency (TD), and Reentrancy (RE). These vulnerability types typically exhibit specific code patterns. For instance, reentrancy vulnerabilities often involve the use of the `call.value()` function to send tokens. Theoretically, static analysis tools based on code patterns should be able to identify these vulnerabilities with 100% accuracy. Table 7 presents the experimental results of DeepDesc on this dataset.

DeepDesc demonstrated excellent performance in identifying these MABs, achieving near-perfect accuracy. For DC vulnerabilities, DeepDesc produced only 1 FP out of 500 negative samples and 1 FN out of 196 positive samples. When dealing with IO and TD vulnerabilities, there were no misclassifications across all samples. For RE vulnerabilities, DeepDesc had no FP in 500 negative samples and only 1 FN in 273 positive samples. These results prove that DeepDesc's deep learning approach is effective in identifying vulnerability types with sufficient dataset availability, providing a foundation for its potential in detecting MUBs.

4.3.2 MUBs

The primary objective of DeepDesc is to identify MUBs. This section will present and analyze the experimental results of DeepDesc in this domain. We evaluated three types of MUBs: Price Oracle Manipulation (PM), Atomicity Violations (AV), and Inconsistent State Updates (ISU). Table 8 shows the detailed experimental results.

For PM vulnerability detection, DeepDesc identified 495 TN and 5 FP out of 500 negative samples, while detecting 132 TP and 6 FN among the positive samples. For AV and ISU vulnerabilities, DeepDesc performed excellently in identifying negative samples, achieving perfect recognition (TN=500, FP=0). In positive samples, AV vulnerability detected 27 TP and 7 FN, while ISU vulnerability detected 29 TP and 9 FN.

Table 7 MABs detection results

Datasets	Metrics	DC	IO	TD	RE
Negative Sample	TN	499	500	500	500
	FP	1	0	0	0
Positive Sample	TP	195	275	349	272
	FN	1	0	0	1

Table 8 MUBs detection results

	Metrics	AV	ISU	PM
Performance	Accuracy	0.99	0.98	0.98
	Precision	1.00	1.00	0.96
	Recall	0.81	0.77	0.95
	F1 Score	0.88	0.87	0.96
Negative Sample	TN	500	500	495
	FP	0	0	5
Positive Sample	TP	27	29	132
	FN	7	9	6

DeepDesc demonstrated extremely high precision across all three vulnerability types. In terms of recall, PM vulnerability stood out with a recall of 0.96, significantly higher than AV's 0.81 and ISU's 0.77. This difference may stem from the varying dataset sizes: the PM dataset is the largest, with 132 samples, whereas AV and ISU have only 27 and 29 samples, respectively.

These results suggest that dataset size plays a crucial role in detection accuracy, while also highlighting the potential of DeepDesc. As the AV, ISU, and other vulnerability datasets continue to grow, it is expected that DeepDesc's recall rates will further improve.

4.4 Comparison with Existing Tools

The four MABs discussed in this paper (reentrancy, integer overflow, delegate call vulnerability, and timestamp dependency) all have fixed code patterns or control flow patterns, which existing static analysis tools and pattern-matching methods can fully identify. Therefore, this section focuses only on MUBs. Existing tools (Feist et al. 2019; Tsankov et al. 2018; Tikhomirov et al. 2018; Luu et al. 2016; Jiang et al. 2018) are almost incapable of examining the MUBs discussed in this paper. David et al. (2023) utilized LLMs for vulnerability detection, which theoretically can identify MUBs. As their tool is not publicly available, we directly cite the experimental results from their paper: using the GPT-4 model achieved 4.14% accuracy, 43.84% recall, and 7.57% F1 score. Our results shown in Table 2 are also consistent with their experimental findings, with GPT-3.5 achieving only 15% and 19% accuracy in yes-or-no answers when detecting complex logical vulnerabilities such as PM.

On our MUB benchmark (Table 9), DeepDesc markedly outperforms both direct LLM prompting and GPTLens(now based on GPT-5) (Hu et al. 2023). Specifically, DeepDesc achieves P/R/F1 of 1.00/0.81/0.88 on AV, 1.00/0.77/0.87 on ISU, and 0.96/0.95/0.96 on PM, whereas GPTLens attains F1 of 0.38, 0.34, and 0.37, respectively. Modern LLMs used

Table 9 Comparison with LLMs, GPTLens and DeepDesc

Module	AV			ISU			PM		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score	Precision	Recall	F1 Score
DeepSeekV3.1	0.15	0.08	0.10	0.07	0.03	0.04	0.15	0.10	0.12
GPT-5	0.19	0.42	0.26	0.26	0.33	0.29	0.22	0.25	0.23
Claude-4.0	0.23	0.33	0.27	0.24	0.22	0.23	0.18	0.32	0.23
GPTLens	0.32	0.47	0.38	0.30	0.38	0.34	0.34	0.41	0.37
DeepDesc	1.00	0.81	0.88	1.00	0.77	0.87	0.96	0.95	0.96

directly (DeepSeekV3.1, GPT-5, Claude-4.0) yield only 0.10–0.29 F1 across these tasks. These results indicate that, for detecting MUBs, DeepDesc substantially outperforms existing LLM-based approaches.

4.5 Effectiveness Analysis of Each Module

This section examines the effectiveness of various DeepDesc modules, using MUBs as an example.

4.5.1 MLB

As shown in Table 10, we compare the performance of three approaches: using code only, using code descriptions only, and the MLB (CD) method, which fuses code and descriptions.

For AV vulnerabilities, the MLB (CD) method achieves an F1 score of 0.85, matching the result of the code-only approach, while its precision (0.93) exceeds that of the code-only (0.92) and code descriptions-only (0.81) methods. Interestingly, the MLB (CD) method's recall is lower than the code-only approach, possibly due to random error from the small sample size (only 34 positive samples). In the case of ISU vulnerability, the MLB (CD) method outperforms the other two approaches across all three metrics, with a particularly high precision of 0.96, significantly surpassing the code-only (0.84) and code descriptions-only (0.89) results. For PM, the MLB (CD) method also achieves the best results in all three metrics.

The code descriptions-only method performs well in some cases (e.g., ISU and PM precision), indicating that LLM-generated code descriptions contain valuable information. These findings support our hypothesis that combining code with high-level semantic descriptions provides more comprehensive information, enhancing vulnerability detection performance. In most cases, the MLB (CD) method achieves the best or near-best detection results, demonstrating its effectiveness in information fusion.

4.5.2 RAG

The abbreviations in Table 11 are as follows: MLB(CD) represents the fusion of code and GPT-3.5 generated code descriptions; MLB(CD+FT) indicates the fusion of code and code descriptions generated by a fine-tuned GPT-3.5; MLB(CD+RAG) refers to the fusion of code and code descriptions generated by a GPT-3.5 enhanced with RAG.

As shown in Table 11, we compared the performance of MLB(CD), MLB(CD+FT), and MLB(CD+RAG) in detecting MUBs. The results indicate that the RAG method generally performs the best. For AV vulnerability, the MLB(CD+RAG) method outperforms the other two methods across all metrics, with a precision of 1.00, recall of 0.81, and an F1

Table 10 Precision, F1 Score and Recall of Code, Code description, and MLB (CD)

Module	AV			ISU			PM		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score	Precision	Recall	F1 Score
Code	0.92	0.83	0.85	0.84	0.71	0.76	0.88	0.87	0.87
Code Description	0.81	0.72	0.74	0.89	0.67	0.75	0.90	0.85	0.87
MLB (CD)	0.93	0.79	0.85	0.96	0.72	0.81	0.94	0.89	0.92

Table 11 Precision, F1 Score and Recall of MLB(CD), MLB(CD+FT) and MLB(CD+RAG)

Module	AV			ISU			PM		
	Precision	Recall	F1 Score	Precision	Recall	F1 Score	Precision	Recall	F1 Score
MLB (CD)	0.93	0.79	0.85	0.96	0.72	0.81	0.94	0.89	0.92
MLB (CD+FT)	0.96	0.70	0.78	0.93	0.58	0.71	0.96	0.92	0.94
MLB (CD+RAG)	1.00	0.81	0.88	1.00	0.77	0.87	0.96	0.95	0.96

score of 0.88, all surpassing the baseline MLB(CD) method. In detecting ISU vulnerability, MLB(CD+RAG) also shows outstanding performance. It achieves a precision of 1.00, recall of 0.77, and an F1 score of 0.87, significantly exceeding the other two methods. For PM, the performance of the three methods is closer, but MLB(CD+RAG) still has a slight edge. It matches MLB(CD+FT) in precision (0.96) but scores highest in recall (0.95) and F1 score (0.96).

Unexpectedly, the fine-tuned LLMs (CD+FT) performed worse overall than the baseline MLB(CD) method. We posit that the primary cause is catastrophic forgetting (Luo et al. 2023; Ho et al. 2024; Uddin et al. 2025). Our fine-tuning used only 794 Code4Rena reports; such limited supervision can bias the model parameters toward the training domain while simultaneously eroding the model's original general code-understanding ability, leading to lower-quality descriptions for unfamiliar contracts. This phenomenon—a drop in recall and F1 (especially for the ISU type)—is consistent with typical forgetting signatures, in which the model's coverage narrows.

In contrast, the RAG approach introduces external knowledge and reduces reliance on parametric memory, which has been shown to mitigate forgetting and domain shift (Lewis et al. 2020). These results underscore the effectiveness of RAG-enhanced LLMs in producing high-quality code descriptions and improving downstream vulnerability detection performance.

5 Discussion

This section explores the limitations of the DeepDesc method and potential directions for future improvements.

Vulnerability Type Coverage Currently, DeepDesc is capable of detecting only three types of vulnerabilities: Price Oracle Manipulation, Atomicity Violations, and Inconsistent State Updates. This limitation primarily stems from the general scarcity of MUBs datasets, with these three types representing areas where we were able to obtain relatively sufficient data. Notably, for vulnerability types with limited data availability (such as Inconsistent State Updates), DeepDesc's recall rate is only 0.77, lower than the 0.95 achieved for Price Oracle Manipulation. Specifically, the three MUBs differ markedly in positive-sample sizes — AV: 34, ISU: 38, PM: 138 — with corresponding recalls of 0.81, 0.77, and 0.95, respectively, highlighting a clear positive correlation between dataset size and detection recall. In the future, we plan to continuously expand our MUBs dataset to enhance DeepDesc's detection capabilities and broaden its applicability. To achieve this, we are considering leveraging the generative capabilities of LLMs to create more diverse and informative MUB examples. This approach could involve techniques such as back-translation, paraphrasing, or generat-

ing semantically similar new descriptions, thereby moving beyond simple replication to enrich our dataset. Additionally, considering the rapid evolution of the smart contract security landscape, we recognize the need to dynamically update our knowledge base to ensure its effectiveness against new attack vectors. This will be an important part of our future work.

Interpretability While DeepDesc currently outputs binary labels, its generated descriptions can be further operationalized to provide more developer-relevant insights. For span-level vulnerability localization, we plan to align description sentences with concrete code statements via cross-modal attention from the MLB fusion and retrieval match signals, producing the top- k highlighted lines; evaluation will use line- and token-level Precision, Recall, and F1. For severity assessment, we will extend the classifier to predict ordinal severity levels (e.g., High/Medium/Low, following Code4rena (2024) and the SWC guidelines ConsenSys Diligence 2018). These enhancements aim to transform the model's descriptions into precise location annotations and impact estimations, thereby improving interpretability and prioritization value.

6 Related Work

This section explores research relevant to our work. In the field of static analysis, symbolic execution is the most widely applied technique. Oyente (Luu et al. 2016) detects four types of vulnerabilities, including reentrancy attacks, by symbolically traversing all possible execution paths. Securify (Tsankov et al. 2018) combines symbolic execution with Datalog reasoning to check for compliance and violation patterns. Slither (Feist et al. 2019) converts Solidity smart contracts into an intermediate representation called SlithIR, utilizing SSA and a simplified instruction set for analysis.

Dynamic analysis primarily relies on fuzzing techniques (Zhu et al. 2022). ContractFuzzer (Jiang et al. 2018) generates fuzzy inputs based on the ABI specifications of smart contracts. Ityfuzz (Shou et al. 2023), a snapshot-based fuzzing tool, tests smart contracts by creating and storing snapshots of blockchain states.

In machine learning approaches, Xing et al. (2020) introduced a novel slice matrix feature extraction method, identifying vulnerability patterns by segmenting contract opcode sequences. Zhuang et al. (2021) transformed smart contract code into graph structures, leveraging Graph Neural Networks (GNN) for analysis.

Following the advent of ChatGPT, David et al. (2023) confirmed that directly using the GPT-4 model for vulnerability detection yields poor results. However, methods employing LLMs as intermediary tools have shown promise. GPTLENS (Hu et al. 2023) adopts a two-stage adversarial framework of generation and discrimination, utilizing LLMs for vulnerability detection. LLM4Fuzz (Shou et al. 2024) uses large language models to intelligently guide fuzzing tools, prioritizing the exploration of code areas likely to trigger vulnerabilities. AuditGPT (Xia et al. 2024) breaks down complex auditing processes into manageable tasks and designs specific prompts for each type of ERC rule to enhance LLM performance in auditing. GPTScan (Sun et al. 2024) employs GPT to match potentially vulnerable functions based on code-level scenarios and attributes, further guiding GPT to intelligently identify key variables and statements, followed by verification through static confirmation.

7 Conclusion

This paper proposes a novel smart contract vulnerability detection method called DeepDesc, which combines the semantic understanding capabilities of LLMs with the classification abilities of deep learning to detect MUBs that traditional tools struggle to identify. DeepDesc leverages LLMs enhanced by RAG to generate high-quality code descriptions that capture the semantic information within the code. It then utilizes MLB to fuse code embeddings and description embeddings, achieving a more comprehensive feature representation for deep learning classification. Experimental results demonstrate that DeepDesc is highly effective in detecting MUBs. Future work will expand DeepDesc's support for more types of logical vulnerabilities and further enhance its accuracy by enlarging the dataset.

Author Contributions Tao Tan collected the data, conducted the experiments, performed the analysis, and drafted the manuscript. Xiao Chen designed the study and contributed to the writing and revision of the manuscript.

Funding Not applicable.

Data Availability Statements The source code, raw data, and intermediate results used in this study are publicly available at <https://github.com/ttan0091/DeepDesc>.

Declarations

Ethical approval Not applicable.

Informed consent Not applicable.

Conflict of Interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Clinical trial number Not applicable.

References

- Buterin V et al (2014) A next-generation smart contract and decentralized application platform. White Paper 3(37):2–1
- Chen X, Li C, Wang D, Wen S, Zhang J, Nepal S, Xiang Y, Ren K (2019) Android HIV: a study of repackaging malware for evading machine-learning detection. *IEEE Trans Inf Forensics Secur* 15:987–1001
- Chu H, Zhang P, Dong H, Xiao Y, Ji S, Li W (2023) A survey on smart contract vulnerabilities: data sources, detection and repair. *Inf Softw Technol* 159:107221
- Code4rena (2024) Code4rena. <https://code4rena.com/>. Accessed 26 Aug 2025
- ConsenSys Diligence (2018) Smart contract weakness classification (swc) registry. <https://swcregistry.io>. Accessed 23 Aug 2025
- David I, Zhou L, Qin K, Song D, Cavallaro L, Gervais A (2023) Do you still need a manual smart contract audit? *arXiv:2306.12338*
- DefiHacks (2023) DefiHacks dataset. <https://wooded-meter-1d8.notion.site/0e85e02c5ed34df3855ea9f3ca40f53b?v=22e5e2c506ef4caeb40b4f78e23517ee>. Accessed 22 Aug 2025
- Deng Z, Guo Y, Han C, Ma W, Xiong J, Wen S, Xiang Y (2025a) Ai agents under threat: a survey of key security challenges and future pathways. *ACM Comput Surv* 57(182):1–36. <https://doi.org/10.1145/3716628>

- Deng Z, Ma W, Han Q, Zhou W, Zhu X, Wen S, Xiang Y (2025b) Exploring deepseek: a survey on advances, applications, challenges and future directions. *IEEE/CAA J Automatica Sinica* 12(5):872–893. <https://doi.org/10.1109/JAS.2025.125498>
- Deng Z, Sun R, Xue M, Ma W, Wen S, Nepal S, Yang X (2025c) Hardening llm fine-tuning: from differentially private data selection to trustworthy model quantization. *IEEE Trans Inf Forensics Secur* 20:7211–7226. <https://doi.org/10.1109/TIFS.2025.3581103>
- Deng Z, Sun R, Xue M, Wen S, Camtepe S, Nepal S, Xiang Y (2025d) Leakage-resilient and carbon-neutral aggregation featuring the federated ai-enabled critical infrastructure. *IEEE Trans Dependable Secure Comput* 22(4):3661–3675. <https://doi.org/10.1109/TDSC.2025.3532957>
- Feist J, Grieco G, Groce A (2019) Slither: a static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WET-SEB). IEEE, pp 8–15
- Ho S, Liu M, Du L, Gao L, Xiang Y (2024) Prototype-guided memory replay for continual learning. *IEEE Trans Neural Netw Learning Syst* 35(8):10973–10983. <https://doi.org/10.1109/TNNLS.2023.3246049>
- Hu S, Huang T, Ilhan F, Tekin SF, Liu L (2023) Large language model-powered smart contract vulnerability detection: new perspectives. In: 2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA). IEEE, pp 297–306
- Jiang B, Liu Y, Chan WK (2018) Contractfuzzer: fuzzing smart contracts for vulnerability detection. In: Proceedings of the 33rd ACM/IEEE international conference on automated software engineering, pp 259–269
- Jiao T, Xu Z, Qi M, Wen S, Xiang Y, Nan G (2024) A survey of ethereum smart contract security: attacks and detection. *Distributed Ledger Technologies Research and Practice* 3(3):1–28
- Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Wt Y, Rocktäschel T et al (2020) Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv Neural Inf Process Syst* 33:9459–9474
- Luo Y, Yang Z, Meng F, Li Y, Zhou J, Zhang Y (2023) An empirical study of catastrophic forgetting in large language models during continual fine-tuning. [arXiv:2308.08747](https://arxiv.org/abs/2308.08747)
- Luu L, Chu DH, Olickel H, Saxena P, Hobor A (2016) Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security, pp 254–269
- MetaTrustLabs (2023a) GPTScan-DefiHacks. <https://github.com/MetaTrustLabs/GPTScan-DefiHacks>. Accessed 22 Aug 2025
- MetaTrustLabs (2023b) GPTScan-Top200. <https://github.com/MetaTrustLabs/GPTScan-Top200>. Accessed 22 Aug 2025
- MetaTrustLabs (2023c) GPTScan-Web3Bugs. <https://github.com/MetaTrustLabs/GPTScan-Web3Bugs>. Accessed 22 Aug 2025
- Qian P, Liu Z, Yin Y, He Q (2023) Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode. In: Proceedings of the ACM web conference 2023, pp 2220–2229
- Shou C, Liu J, Lu D, Sen K (2024) Llm4fuzz: guided fuzzing of smart contracts with large language models. [arXiv:2401.11108](https://arxiv.org/abs/2401.11108)
- Shou C, Tan S, Sen K (2023) Ityfuzz: snapshot-based fuzzer for smart contract. In: Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis, pp 322–333
- Sun Y, Wu D, Xue Y, Liu H, Wang H, Xu Z, Xie X, Liu Y (2024) Gptscan: detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th international conference on software engineering, pp 1–13
- Tikhomirov S, Voskresenskaya E, Ivanitskiy I, Takhaviev R, Marchenko E, Alexandrov Y (2018) Smart-check: static analysis of ethereum smart contracts. In: Proceedings of the 1st international workshop on emerging trends in software engineering for blockchain, pp 9–16
- Tsankov P, Dan A, Drachsler-Cohen D, Gervais A, Buenzli F, Vechev M (2018) Securify: practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC conference on computer and communications security, pp 67–82
- Uddin MP, Xiang Y, Hasan M, Bai J, Zhao Y, Gao L (2025) A systematic literature review of robust federated learning: issues, solutions, and future research directions. *ACM Comput Surv* 57(10). <https://doi.org/10.1145/3727643>
- Ullah S, Han M, Pujar S, Pearce H, Coskun A, Stringhini G (2024) LLMs cannot reliably identify and reason about security vulnerabilities (yet?): a comprehensive evaluation, framework, and benchmarks. In: IEEE symposium on security and privacy
- Wan Y, Qu Y, Ni W, Xiang Y, Gao L, Hossain E (2024) Data and model poisoning backdoor attacks on wireless federated learning, and the defense mechanisms: a comprehensive survey. *IEEE Commun Surv Tutor* 26(3):1861–1897. <https://doi.org/10.1109/COMST.2024.3361451>
- Xia S, Shao S, He M, Yu T, Song L, Zhang Y (2024) Auditgpt: auditing smart contracts with chatgpt. [arXiv:2404.04306](https://arxiv.org/abs/2404.04306)

- Xing C, Chen Z, Chen L, Guo X, Zheng Z, Li J (2020) A new scheme of vulnerability analysis in smart contract with machine learning. *Wireless Networks*, pp 1–10
- Zhang Z (2023) Web3Bugs. <https://github.com/ZhangZhuoSJTU/Web3Bugs>. Accessed 22 Aug 2025
- Zhang Z, Zhang B, Xu W, Lin Z (2023) Demystifying exploitable bugs in smart contracts. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, pp 615–627
- Zhao Y, Qu Y, Xiang Y, Uddin Mp, Peng D, Gao L (2025) A comprehensive survey on edge data integrity verification: fundamentals and future trends. *ACM Comput Surv* 57(1). <https://doi.org/10.1145/3680277>
- Zhou W, Zhu X, Han QL, Li L, Chen X, Wen S, Xiang Y (2025) The security of using large language models - a survey with emphasis on chatgpt. *IEEE/CAA J Automatica Sinica* 12(1):1–26. <https://doi.org/10.1109/JAS.2024.124983>
- Zhuang Y, Liu Z, Qian P, Liu Q, Wang X, He Q (2021) Smart contract vulnerability detection using graph neural networks. In: Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence, pp 3283–3290
- Zhu X, Wen S, Camtepe S, Xiang Y (2022) Fuzzing: a survey for roadmap. *ACM Comput Surv* 54(11s). <https://doi.org/10.1145/3512345>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.